

Prompt and Format Robustness in LLM Evaluation

Capstone Advisor: Kai-Wei Chang

Allen Ma

M.S. Computer Science

University of California, Los Angeles

Abstract

Large language model (LLM) evaluations often report a single score obtained from a single prompt template, but recent work suggests that such scores can be sensitive to seemingly minor prompt-design choices. This project studies prompt and output-format robustness on two structured tasks: TREC-6 question classification and GSM8K grade-school math. I evaluate two modern OpenAI models, `gpt-5-nano` and `gpt-5-mini`, on 100 frozen examples per task using eight semantically equivalent prompt variants formed by crossing two instruction wordings with four output-format constraints. Each raw model output is scored twice: once with strict exact-format scoring and once with a deterministic equivalence-aware scorer that extracts and normalizes harmless answer-format variations. The main results show moderate prompt sensitivity: prompt accuracy ranges from 2–8 percentage points depending on task, model, and scorer. Equivalence-aware scoring reduces GSM8K prompt sensitivity by 1 percentage point for both models and increases average GSM8K accuracy by recovering benign numeric formatting differences such as `5.00` versus `5`. TREC-6 shows essentially no strict-versus-equivalence scoring gap, because model outputs are typically valid labels or genuinely wrong labels. As a secondary validation, a `gpt-5.4` judge audited all 326 outputs that failed strict scoring and agreed exactly with the deterministic equivalence-aware scorer. These findings support the view that prompt sensitivity is present but modest in this controlled setting, and that scoring artifacts matter most when task outputs admit harmless surface-form variation.

1 Introduction

LLM evaluation is often treated as if a model-task pair has a stable scalar performance value. In practice, prompting-based evaluation introduces an additional choice: the prompt template. Prior work has shown that prompt formatting and instruction wording can substantially change measured model performance even when the underlying task and examples remain fixed [5, 4]. This makes single-prompt evaluation fragile: a reported score may partly reflect the arbitrary prompt selected by the evaluator.

Recent work further argues that some measured prompt sensitivity may be an evaluation artifact rather than pure model brittleness. In particular, rigid exact-match or heuristic scoring can mark semantically correct outputs as wrong when the model uses a different but equivalent surface form [2]. For example, if the gold answer is `5`, the output `The answer is 5.00` may be semantically correct but fail a strict string match. If different prompts systematically elicit different output styles, apparent prompt sensitivity may partly reflect scoring brittleness.

This project tests that issue in a small, controlled empirical setting. I evaluate two modern OpenAI models on two structured tasks under a fixed family of semantically equivalent prompts. The prompts vary along two axes: instruction wording and output-format constraint. I then score the same raw outputs using both strict exact-format scoring and a rule-based equivalence-aware scorer. This design separates two questions:

1. How much does measured accuracy vary across semantically equivalent prompt variants?
2. How much of that variation remains after harmless output-format differences are normalized?

The goal is not to introduce a new benchmark or maximize model coverage. The goal is to run a clear and reproducible prompt-robustness experiment that distinguishes model/output variation from evaluator brittleness.

2 Related Work

Prompt-format sensitivity. Sclar et al. [5] study LLM sensitivity to meaning-preserving prompt-format choices. They show that plausible formatting changes can cause large performance swings and argue that evaluations should report performance ranges across prompt formats rather than a single arbitrarily chosen prompt. This project follows that basic framing but uses a smaller modern-model retest with explicitly controlled prompt families.

Multi-prompt evaluation. Mizrahi et al. [4] argue that single-prompt LLM evaluation is unreliable and propose evaluating over multiple semantically equivalent instruction templates. Their definition of an instruction template as a string with placeholders for fixed input examples matches the design used here: the dataset examples are held fixed while the prompt wrapper changes.

Evaluation artifacts. Hua et al. [2] revisit prompt sensitivity and argue that much of the reported effect may come from brittle evaluation, including rigid answer matching and heuristic scoring. They show that using more semantic evaluation, including LLM-as-judge evaluation, can reduce measured performance variance. This motivates the main scoring ablation in this project: strict scoring versus equivalence-aware scoring.

LLM-as-judge evaluation. LLM-as-judge methods have been used as scalable evaluators for model outputs, especially for open-ended generation. Zheng et al. [6] study strong LLMs as judges in MT-Bench and Chatbot Arena and discuss both their utility and limitations. In this project, LLM judging is used only as a secondary audit of the deterministic equivalence-aware scorer, not as the primary evaluator.

3 Experimental Setup

3.1 Tasks and Datasets

I use two structured tasks. The first is TREC-6 question classification [3], a six-way classification task over coarse answer types. The labels are ABBR, ENTY, DESC, HUM, LOC, and NUM. The second is GSM8K [1], a grade-school math word-problem benchmark with final numeric answers.

For each task, I sample 100 examples from the test split using a fixed seed and freeze the sampled examples to disk before inference. All models and prompts use the same frozen examples. Table 1 summarizes the task setup.

3.2 Models

The main experiment evaluates two OpenAI Responses API model IDs:

- `gpt-5-nano`

Task	Type	Output	Examples
TREC-6	Question classification	One of 6 labels	100
GSM8K	Math reasoning	Final numeric answer	100

Table 1: Tasks used in the main experiment.

- `gpt-5-mini`

The model-selection goal was not to benchmark the strongest available systems. Instead, the goal was to use modern API models that are competent enough for the tasks while still leaving room to observe prompt-induced variation. If a model is too weak, prompt sensitivity can be hard to distinguish from general task failure. If a model is near-perfect, ceiling effects can hide sensitivity. The selected models operate in a useful middle-high regime: they achieve high average accuracy, but still show enough errors and prompt-level variance to study robustness.

The experiment was run in May 2026. I did not set a temperature parameter, because the evaluated GPT-5-family models rejected that parameter in initial smoke tests. The main run used provider-default reasoning/decoding behavior. The initial main run used a 2,048 output-token cap; 14 incomplete `gpt-5-nano`/GSM8K records hit that cap and were retried with a 4,096 cap. The final clean output set contains 3,200 completed responses and no empty outputs.

3.3 Prompt Family

The prompt design uses a controlled 2×4 factorial structure:

- two instruction wordings: direct and formal;
- four output formats: answer only, sentence, XML-style tagged answer, and JSON.

This gives eight prompt variants per task. The dataset input itself is never paraphrased; only the surrounding instruction and requested output format change. This keeps the task instance and gold answer fixed across prompts.

Prompt ID	Description
<code>direct_answer_only</code>	Direct wording, bare answer
<code>direct_sentence</code>	Direct wording, “The answer is ...” sentence
<code>direct_tagged</code>	Direct wording, <code><answer>...</answer></code> format
<code>direct_json</code>	Direct wording, JSON answer object
<code>formal_answer_only</code>	Formal wording, bare answer
<code>formal_sentence</code>	Formal wording, sentence answer
<code>formal_tagged</code>	Formal wording, tagged answer
<code>formal_json</code>	Formal wording, JSON answer object

Table 2: The eight prompt variants used for each task.

3.4 Inference Matrix

For each task, model, example, and prompt variant, I run one inference call:

$$2 \text{ tasks} \times 100 \text{ examples} \times 8 \text{ prompts} \times 2 \text{ models} = 3,200 \text{ outputs.}$$

Each output record stores the task, example ID, model ID, prompt ID, raw prompt, raw model output, response status, token usage, and timestamp. These raw outputs are then scored without rerunning inference.

4 Scoring Methods

The scoring design is central to the experiment. The same raw model output is scored by multiple evaluators.

4.1 Strict Scoring

Strict scoring requires exact compliance with the requested output format and exact equality with the gold answer. For answer-only prompts, the output must exactly equal the gold answer. For sentence prompts, the output must exactly equal `The answer is GOLD..` For tagged prompts, it must exactly equal `<answer>GOLD</answer>`. For JSON prompts, it must be valid JSON with exactly one key, `answer`, whose value stringifies to the gold answer.

This intentionally rigid scorer models brittle evaluation pipelines that punish harmless formatting differences.

4.2 Deterministic Equivalence-Aware Scoring

The equivalence-aware scorer is a deterministic extraction-and-normalization scorer. It is not a general semantic evaluator. It handles common benign output variations induced by the prompt family.

For TREC-6, the scorer attempts to extract an answer from JSON, XML-style tags, an “answer is” sentence, or the raw output. It canonicalizes labels by uppercasing, removing punctuation, and mapping aliases to canonical labels. For example, `location` and `place map` map to `LOC`, and `person` maps to `HUM`.

For GSM8K, the scorer extracts a final numeric answer from JSON, tags, an “answer is” sentence, or the last number in the output. It normalizes commas, currency symbols, trailing periods, whitespace, and decimal-equivalent forms using exact decimal comparison. Thus `5`, `5.00`, `$5.00`, and `5,000` are handled as numeric forms rather than raw strings.

4.3 LLM-Judge Audit

As a secondary validation, I run an LLM-judge audit on all outputs that failed strict scoring. This subset contains 326 outputs. The judge model is `gpt-5.4`. The judge receives task-specific instructions, the original input, the gold answer, and the model output. For TREC-6, the prompt includes label definitions. For GSM8K, the prompt instructs the judge to evaluate final numeric equivalence and ignore harmless formatting differences.

This audit is not treated as the main scorer. Instead, it checks whether the deterministic equivalence-aware scorer is too permissive or too conservative on exactly the cases where strict scoring fails.

5 Metrics

For each task, model, prompt, and scorer, I compute accuracy over the 100 frozen examples. The primary robustness metric is prompt sensitivity range:

$$\text{Sensitivity}(t, m, s) = \max_{p \in P} \text{Acc}(t, m, p, s) - \min_{p \in P} \text{Acc}(t, m, p, s),$$

Task	Model	Scorer	Avg.	Min	Max	Std.	Sens.
GSM8K	gpt-5-mini	Strict	91.1	90.0	93.0	1.0	3.0
GSM8K	gpt-5-mini	Equiv.	96.2	95.0	97.0	0.7	2.0
GSM8K	gpt-5-nano	Strict	92.9	90.0	97.0	2.2	7.0
GSM8K	gpt-5-nano	Equiv.	95.0	92.0	98.0	1.7	6.0
TREC-6	gpt-5-mini	Strict	88.0	84.0	89.0	1.7	5.0
TREC-6	gpt-5-mini	Equiv.	88.0	84.0	89.0	1.7	5.0
TREC-6	gpt-5-nano	Strict	87.2	84.0	92.0	2.6	8.0
TREC-6	gpt-5-nano	Equiv.	87.2	84.0	92.0	2.6	8.0

Table 3: Main results. Values are percentages. Avg. is average accuracy across the eight prompt variants. Std. is the standard deviation of prompt accuracies. Sens. is max prompt accuracy minus min prompt accuracy.

where t is task, m is model, s is scorer, and P is the set of eight prompt variants. To measure scoring artifacts, I compute:

$$\text{ArtifactGap}(t, m) = \text{Sensitivity}_{\text{strict}}(t, m) - \text{Sensitivity}_{\text{equiv}}(t, m).$$

A positive artifact gap means prompt sensitivity shrinks under equivalence-aware scoring, suggesting that some strict-scoring sensitivity came from evaluation brittleness.

6 Results

6.1 Main Accuracy and Sensitivity

Table 3 reports average accuracy across prompts, minimum and maximum prompt accuracy, standard deviation across prompt accuracies, and prompt sensitivity range.

Prompt sensitivity is present across both tasks and both models. The observed sensitivity ranges from 2 to 8 percentage points. The smaller gpt-5-nano model is generally more prompt-sensitive than gpt-5-mini. On GSM8K, gpt-5-nano has a 7-point strict sensitivity range, compared to 3 points for gpt-5-mini. On TREC-6, gpt-5-nano has an 8-point range, compared to 5 points for gpt-5-mini.

The largest sensitivity appears for gpt-5-nano on TREC-6, where prompt accuracies range from 84% to 92%. This suggests that even for a simple structured classification task, the measured performance of a fixed model can change noticeably across prompt variants.

The average accuracy columns in Table 3 provide useful context for model quality. The two models have similar average accuracy in several settings; for example, under strict scoring on GSM8K, gpt-5-nano is slightly higher on average than gpt-5-mini, while under equivalence-aware scoring gpt-5-mini is higher. On TREC-6, gpt-5-mini is only slightly higher on average. The more consistent difference is not average accuracy, but robustness: gpt-5-nano has larger prompt-sensitivity ranges and larger prompt-accuracy standard deviations on both tasks. This suggests that model capability may affect robustness even when average accuracy is close, although this should be interpreted cautiously because only two models are evaluated.

6.2 Effect of Scoring Method

The scoring ablation has a clear but task-dependent effect. On GSM8K, equivalence-aware scoring increases average accuracy and slightly reduces prompt sensitivity. For gpt-5-mini, average GSM8K accuracy

increases from 91.1% under strict scoring to 96.2% under equivalence-aware scoring. For `gpt-5-nano`, it increases from 92.9% to 95.0%. In both cases, the prompt sensitivity range shrinks by 1 percentage point.

On TREC-6, strict and equivalence-aware results are identical. The reason is straightforward: model outputs usually either match the requested label format or choose a genuinely wrong label. There are few harmless surface-form variations for the equivalence-aware scorer to recover.

Task	Model	Strict Sens.	Equiv. Sens.	Artifact Gap
GSM8K	<code>gpt-5-mini</code>	3.0	2.0	1.0
GSM8K	<code>gpt-5-nano</code>	7.0	6.0	1.0
TREC-6	<code>gpt-5-mini</code>	5.0	5.0	0.0
TREC-6	<code>gpt-5-nano</code>	8.0	8.0	0.0

Table 4: Prompt sensitivity artifact gap. Values are percentage points.

6.3 Prompt-Level Accuracy

Figure 1 and Figure 2 show prompt-level accuracies for each task/model pair. These figures make two patterns visible. First, GSM8K exhibits a consistent vertical separation between strict and equivalence-aware scoring, especially for `gpt-5-mini`. Second, TREC-6 prompt sensitivity is almost entirely model/output driven rather than scorer driven, since strict and equivalence-aware bars overlap.

Table 5 averages each prompt variant across the four task/model settings. This aggregation is not meant to identify a universally optimal prompt, since it collapses across different tasks and models. It is still useful as a descriptive check on whether some prompt styles are consistently weaker or stronger in this experiment.

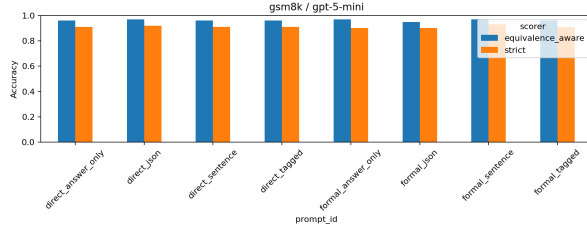
Prompt Variant	Strict Avg.	Equiv. Avg.	Delta
<code>direct_sentence</code>	88.2	90.0	+1.8
<code>direct_answer_only</code>	89.0	91.0	+2.0
<code>formal_answer_only</code>	89.0	91.5	+2.5
<code>formal_json</code>	89.0	90.8	+1.8
<code>direct_json</code>	89.5	91.5	+2.0
<code>direct_tagged</code>	90.2	92.0	+1.8
<code>formal_sentence</code>	91.2	92.5	+1.3
<code>formal_tagged</code>	92.2	93.8	+1.6

Table 5: Average prompt-variant accuracy across both tasks and both models. Values are percentages. Delta is equivalence-aware average accuracy minus strict average accuracy.

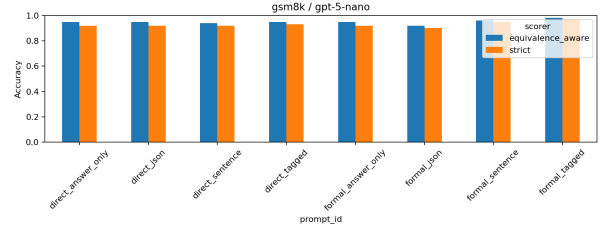
The aggregated trend is that `formal_tagged` is the strongest prompt variant and `direct_sentence` is the weakest under both scoring methods. This pattern should not be overinterpreted, but it suggests that the exact form of the output constraint matters. In this setup, the tagged format may provide a clear answer slot without requiring JSON syntax or a natural-language sentence wrapper. The formal wording is also slightly better on average than the direct wording, though the effect is small and may depend on the task/model pair.

6.4 Scorer Recoveries

Across all 3,200 outputs, 326 fail strict scoring. The equivalence-aware scorer recovers 58 of these. All 58 recoveries occur on GSM8K. Of these, 41 are from `gpt-5-mini` and 17 are from `gpt-5-nano`. Typical recoveries involve numeric formatting:

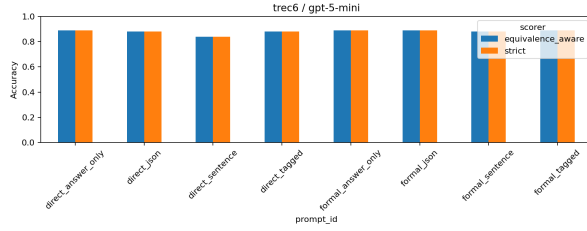


(a) GSM8K, gpt-5-mini

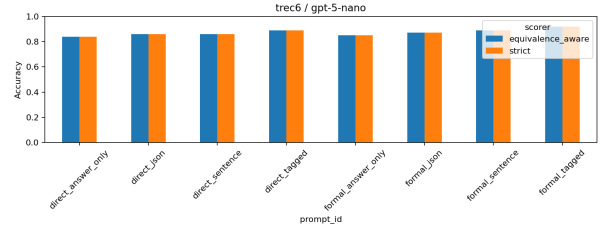


(b) GSM8K, gpt-5-nano

Figure 1: Prompt-level accuracy on GSM8K.



(a) TREC-6, gpt-5-mini



(b) TREC-6, gpt-5-nano

Figure 2: Prompt-level accuracy on TREC-6.

- gold 5, output 5.00;
- gold 5,600, output 5600;
- gold 2, output \$2.00;
- gold 826, output 826.00.

These are not reasoning differences. They are evaluation-format artifacts. Strict scoring marks them wrong because the string differs from the gold answer or requested format, while the equivalence-aware scorer correctly treats them as numerically equivalent.

7 LLM-Judge Audit

The gpt-5.4 judge audit covers every output with `strict_correct = false`. This includes both outputs recovered by the rule-based equivalence-aware scorer and outputs rejected by both deterministic scorers.

Task	Model	Judged	Rule Equiv.	Judge Equiv.	Agreement
GSM8K	gpt-5-mini	71	41	41	100%
GSM8K	gpt-5-nano	57	17	17	100%
TREC-6	gpt-5-mini	96	0	0	100%
TREC-6	gpt-5-nano	102	0	0	100%

Table 6: LLM-judge audit over all strict-false outputs.

The judge produces no parse failures and agrees exactly with the deterministic equivalence-aware scorer. It does not recover any additional cases rejected by the rule-based scorer, and it does not reject any cases recovered by the rule-based scorer. This supports the adequacy of the deterministic scorer for these two structured short-answer tasks.

8 Error Analysis

GSM8K. GSM8K errors mostly fall into two categories. The first is genuine wrong-answer behavior, where the model outputs an incorrect final number. The second is harmless numeric formatting, where strict scoring fails but equivalence-aware scoring succeeds. The latter category is exactly the scoring artifact targeted by this project.

The prompt sensitivity on GSM8K is modest but nonzero. For `gpt-5-nano`, strict prompt accuracy ranges from 90% to 97%. The best-performing prompt is `formal_tagged`, while `formal_json` is lowest. This suggests that output constraints can interact with model behavior, although the effect is not large in this setting.

TREC-6. TREC-6 errors are primarily label confusions. For example, questions whose answer type is an entity can be confused with descriptions, and location-seeking questions can be confused with entity labels when the target answer is a named object such as a mountain range. Because the output space is a small label set, the equivalence-aware scorer has little to recover beyond minor label aliases. This explains why strict and equivalence-aware scoring produce identical TREC-6 results.

Token-cap cleanup. During the first main run, 14 `gpt-5-nano`/GSM8K responses hit the maximum output-token cap and returned empty outputs. These records were retried with a larger output-token cap and merged into the clean final run. The final reported results use only the cleaned run, which has 3,200 completed outputs and no empty responses.

9 Limitations

This project is intentionally scoped. It uses only two tasks, two OpenAI models, and 100 examples per task. The prompt family is controlled and interpretable, but it is not exhaustive. Other prompt paraphrases, few-shot settings, or reasoning-style prompts could produce different sensitivity patterns.

The equivalence-aware scorer is deterministic and task-specific. This is a strength for reproducibility, but it limits generality. It is appropriate for TREC-6 labels and GSM8K numeric answers, but would not directly transfer to open-ended generation. The LLM-judge audit helps validate the scorer in this setting, but it is not a substitute for human evaluation.

The experiment also uses provider-default reasoning and decoding behavior. This reflects normal API use, but it means hidden reasoning-token behavior is not fully controlled. Finally, because the evaluated models are from a single provider, the results should not be interpreted as a broad cross-provider robustness claim.

10 Conclusion

This project re-tests prompt and output-format sensitivity on modern LLMs in a small controlled setting and adds a scoring-method ablation. The results show moderate prompt sensitivity across both tasks and models. The observed sensitivity is not purely a scoring artifact: prompt-level accuracy still varies after equivalence-aware scoring. However, scoring artifacts are present on GSM8K, where strict matching

undercounts semantically correct numeric outputs with harmless formatting differences. TREC-6 shows no strict-versus-equivalence gap because the outputs are usually canonical labels or genuinely wrong labels.

The LLM-judge audit agrees exactly with the deterministic equivalence-aware scorer on all strict-false cases, suggesting that the rule-based scorer is adequate for these structured tasks. Overall, the results support a pragmatic conclusion: multi-prompt evaluation remains useful, and scoring definitions materially affect measured performance when tasks admit benign output variation.

A Prompt Templates

The exact prompt consists of a wording instruction, the task input, and an output-format instruction.

A.1 Instruction Wordings

TREC-6 direct:

Classify the question into exactly one of these labels: ABBR, ENTY, DESC, HUM, LOC, NUM.

TREC-6 formal:

Assign the coarse question type for the item below. The valid labels are ABBR, ENTY, DESC, HUM, LOC, and NUM.

GSM8K direct:

Solve the math problem.

GSM8K formal:

Determine the final numerical answer for the grade-school math problem below.

A.2 Output-Format Instructions

TREC-6:

- Answer only: Return only the label, with no extra text.
- Sentence: Return exactly one sentence in this form: The answer is <label>.
- Tagged: Return the answer inside XML tags in this form: <answer>LABEL</answer>.
- JSON: Return valid JSON in this form: {"answer": "<label>"}

GSM8K:

- Answer only: Return only the final number, with no extra text.
- Sentence: Return exactly one sentence in this form: The answer is <number>.
- Tagged: Return the answer inside XML tags in this form: <answer>NUMBER</answer>.
- JSON: Return valid JSON in this form: {"answer": "<number>"}

B Scoring Details

The strict scorer is deliberately brittle. It requires exact format compliance and exact answer equality. The equivalence-aware scorer parses JSON, XML-style tags, answer sentences, and raw outputs. For TREC-6, it canonicalizes labels and a small alias set. For GSM8K, it extracts numeric answers and compares exact decimal values after removing commas, currency symbols, trailing periods, and whitespace.

C Reproducibility Artifacts

The final committable results are stored under:

- `reports/results/main_clean/`
- `reports/results/full_outputs/`
- `reports/results/judge_audit_gpt54/`

The full clean output set contains 3,200 records. The LLM-judge audit contains 326 judged records.

References

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [2] Andong Hua, Kenan Tang, Chenhe Gu, Jindong Gu, Eric Wong, and Yao Qin. Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19889–19899, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.1006. URL <https://aclanthology.org/2025.emnlp-main.1006/>.
- [3] Xin Li and Dan Roth. Learning question classifiers. In *COLING 2002: The 19th International Conference on Computational Linguistics*, 2002. URL <https://aclanthology.org/C02-1150/>.
- [4] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? a call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024. doi: 10.1162/tacl.a.00681. URL <https://aclanthology.org/2024.tacl-1.52/>.
- [5] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.11324>.
- [6] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhaghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.